**Lorena Silveira dos Santos**
**2312667**

A. Crie um arquivo C contendo as funções `dump` e `main`, compile-o e execute o programa. Explique o resultado desse programa (qual o tamanho, em bytes, de um inteiro? como esses bytes estão ordenados na memória?). Experimente, se necessário, outros valores para `i`.

```
[c2312667@maria-farinha lab2]$ gcc -Wall -o dump dump.c
[c2312667@maria-farinha lab2]$ ./dump
0x7fff16ac25fc - 10
0x7fff16ac25fd - 27
0x7fff16ac25fe - 00
0x7fff16ac25ff - 00
[c2312667@maria-farinha lab2]$
```

O resultado mostra o endereço de memória que cada byte de um inteiro fica armazenado, com um inteiro ocupando 4 bytes sempre e depois mostra o valor que está armazenado em cada um deles em hexadecimal,

B. Aplique a mesma técnica para ver como um `long` é armazenado.
Ele segue a mesma lógica, porém ocupando 8 bytes

C. Aplique a mesma técnica para ver como um `short` é armazenado.
Segue a mesma lógica, porém ocupando 2 bytes.

D. Aplique a mesma técnica para ver como um `char` é armazenado. Lembre-se que uma variável do tipo `char` armazena um **valor inteiro**. Experimente atribuir a essa variável tanto um *caractere* como o valor do código ASCII correspondente (por exemplo, `'a'` e `97`).

```
0x7ffd7963587f - 61
```

colocando o caractere ‘a’, ele nos entrega o endereço de memória em que foi armazenado e depois o valor de ‘a’ na tabela ASCII convertido para hexadecimal

Agora, quando enviamos direto o valor de ‘a’ na tabela ASCII 97, o resultado mostra esse valor convertido pra hexadecimal

Testando para outros valores:

```
int main (void) {
  char i = '\n';
  dump(&i, sizeof(i));
  return 0;
}
```

```
0x7ffeea3897bf - 0a
```

```
int main (void) {
  char i = 'A';
  dump(&i, sizeof(i));
  return 0;
```

```
[c2312667@maria-farinh
0x7ffebaa93c7f - 41
```

```
int main (void) {
  char i = ' ';
  dump(&i, sizeof(i));
  return 0;
}
```

```
0x7ffc8eb17fbf - 20
```

A mesma técnica pode ser usada para vermos como uma string é armazenada:

é armazenado um caractere por byte tendo sempre um byte a mais que armazena '\0' indicando o fim da string

```
int main (void) {
  char p[] = "7509";
  dump(p, sizeof(p));
  return 0;
}
```

```
[c2312667@maria-farinh
0x7fffd0a54a1b - 37
0x7fffd0a54a1c - 35
0x7fffd0a54a1d - 30
0x7fffd0a54a1e - 39
0x7fffd0a54a1f - 00
```

2.

a. Procure entender como funciona a função `string2num`. O que está acontecendo? O que será o valor de `(*s - '0')` a cada iteração? E o valor de **`a`**?

(*s-'0') faz com que nós obtenhamos o inteiro que equivale aquela string porque os números na tabela ASCII tem um valor específico e são ordenados na ordem crescente, então quando a gente diminui o número que temos pelo primeiro número da tabela que seria '0', nós obtemos o valor real daquele número.

ex: Valor de '0' na tabela ASCII : 48

Valor de '5' na tabela ASCII :  53

53-48 = 5

b. Modifique a função `string2num` do exercício anterior para converter strings contendo numerais escritos em qualquer base entre 2 e 10. A base em que o numeral está escrito deverá ser dada como um segundo parâmetro (do tipo `int`) para a função.

```
C dump.c
#include <ctype.h>
#include <stdio.h>

int string2num (char *s, int base){
  int a = 0;
  for (; *s; s++)
    a = a*base + (*s - '0');
  return a;
}

int main (void) {
  printf("%d\n", string2num("777", 8));
  printf("%d\n", string2num("777", 10));
  return 0;
}
```

```
lorenasilveira@ncc17311:~/Documents/Software_basico/lab2$ ./
511
777
```

c. Modifique novamente a função string2num para aceitar bases maiores que 10. Nesse caso, precisamos de "dígitos" para os valores a partir de 10. Use as letras a-z para isso, com a representando o "dígito" 10 e assim por diante, até z representando o "dígito" 35. (Qual a maior base que podemos usar com esse esquema?)

```c
int string2num (char *s, int base) {

 int a = 0;

 for (; *s; s++){

   if(isdigit(*s)){

     a = a*base + (*s - '0');

   }

   else{

     a = a*base + (*s - 'a' + 10);

   }

 }

 return a;

}



int main (void) {

   printf("%d\n", string2num("1a", 16));

 printf("%d\n", string2num("a09b", 16));

 printf("%d\n", string2num("z09b", 36));

 return 0;

}
```

```
[c2312667@turiacu lab2]$ ./dump
26
41115
```

A maior base que podemos usar com esse esquema é a base 36, pois ele possui conversão apenas para 36 caracteres 0-9 e a-z.

Extra:

```c
#include <stdio.h>


int is_little();

int main() {
    if (is_little()) {
        printf("A memoria eh little_endian\n");
    } else {
        printf("A memoria eh big_endian.\n");
    }

    return 0;
}

int is_little() {
    unsigned int x = 1;
    char *c = (char*)&x;
    printf("%d\n", *c);

    return *c;
}
```

```
lorenasilveira@ncc17311:~/Documents/Software_b
1
A memoria eh little_endian
```

A função is_little armazena o inteiro 1 e depois nós atribuímos o endereço desse número no vetor ***c**. Caso seja um sistema little-endian, **x** seria armazenado na memória como 01 00 00 00 e ***c** acessaria o primeiro byte, contendo 0x01, então o valor de retorno seria 1. Já em um sistema big-endian x seria armazenado na memória como 00 00 00 01, ***c** acessaria o primeiro byte, contendo 0x00, e retornaria 0.

2.

```c
void num2string(char*s, int a, int base){
  int i = 0;
  char temp;
  while(a > 0){
    if(a%base > 9){
      s[i] = ((a%base)+'a'-10);

    }
    else{
      s[i] = ((a%base)+'0');

    }
    a = a/base;
    i++;
  }
  s[i] = '\0';
  for (int j = 0; j < i / 2; j++) {
        temp = s[j];
        s[j] = s[i - j - 1];
        s[i - j - 1] = temp;
    }

}
```

A função num2string percorre o número de trás para a frente dividindo ele e pegando sempre o resto, esse resto quando maior que 10 é convertido para a letra correspondente de acordo com a base e com o número da letra na tabela ASCII e, quando menor que 10, pegamos o valor correspondente a esse número na ASCII. Além disso, ao final do loop inserimos o \0 para indicar o fim da string.

No final, a string fica ao contrário e precisa ser invertida para encontrar o valor correto, o que é feito com o auxílio de uma variável temporária.